

Projet 10

Réalisez une application de recommandation de contenu

Développement d'une solution de recommandation d'articles à partir des interactions des utilisateurs



David Scanu | Octobre 2025



Parcours **AI Engineer**

Sommaire

-  Contexte et problématique métier
-  Analyse du jeu de données
-  4 méthodes de recommandation
-  Résultats comparés
-  Gestion du Cold Start
-  Architecture technique
-  Backend - FastAPI
-  Frontend - Next.js
-  Démonstration
-  Bilan & Livrables





Contexte et problématique métier



My Content : Une start-up innovante

- Mission : Encourager la lecture en recommandant des contenus pertinents et personnalisés
- Focus initial : Recommandation d'articles et de livres aux particuliers
- Vision : Devenir la plateforme de référence pour la découverte de contenus de qualité



Équipe

- **Samia (CEO)** : Vision stratégique et développement business
- **David (CTO)** : Développement technique du MVP et système de recommandation
- **Phase actuelle** : Construction du MVP (Minimum Viable Product)



Choix d'architecture : Serverless

- **Recommandation de Julien** (développeur web consultant) :

Frontend Web ↔ Azure Functions (API) ↔ Système de recommandation



Problématique métier

Défi central :

"Comment recommander du contenu pertinent à nos utilisateurs alors que nous n'avons pas encore de données propres ?"

Solution adoptée :

- Utilisation du dataset public **Globo.com** (portal de nouvelles brésilien)
- Pour **valider notre système** avant le lancement commercial
- **Démontre la scalabilité de notre architecture technique**



Fonctionnalité critique identifiée

User Story prioritaire :

"En tant qu'utilisateur de l'application, je vais recevoir une sélection de cinq articles personnalisés."

Contraintes déterminantes :

- ☒ Gérer l'ajout de nouveaux utilisateurs (cold start)
- ☒ Gérer l'ajout de nouveaux articles (cold start)
- ☒ Architecture scalable pour la croissance future
- ☒ Temps de réponse < 200ms pour l'expérience utilisateur



Analyse des données - Vue d'ensemble



Composition du dataset Globo.com

3 composants principaux :

- Métadonnées articles : **articles_metadata.csv** (364K articles)
- Interactions utilisateurs : **clicks_hour_*.csv** (385 fichiers horaires)
- Embeddings : **articles_embeddings.pickle** (vecteurs 250D, 364MB)



Métriques moyennes

Par utilisateur :

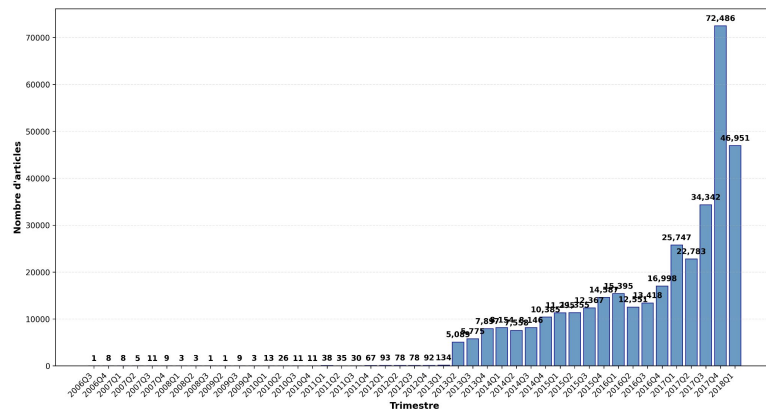
- 9.3 clics/utilisateur en moyenne
- Top utilisateur : 1,232 clics (ID 5890)
- Distribution : 50% < 5 clics, 10% > 20 clics

Par article :

- Articles cohérents :
 - 95% entre 100-300 mots
 - 191 mots en moyenne
- Catégories riches : 461 catégories

Dimension	Valeur	Détails
 Articles totaux	364,047	Catalogue complet 2006-2018
 Utilisateurs uniques	322,897	Base utilisateurs significative
 Clics totaux	2,988,181	~3M interactions
 Catégories	461	Très diversifié (sport, politique, culture...)
 Éditeur	1 (Globo.com)	Source unique mais fiable
 Période	2006-09-27 → 2018-03-13	11.5 ans d'historique

Nombre d'articles publiés par trimestre



Vue d'ensemble des 4 méthodes de recommandation

Approche multi-méthodes complémentaires

Notre système combine **4 stratégies** distinctes pour maximiser pertinence, diversité et robustesse :

Popularité normalisée par âge (+ Boost Nouveauté)

```
Score = (clics × users_uniques) / âge_mois  
if âge < 24h: score *= 1.5 # Boost nouveauté
```

Philosophie : **"Ce qui plaît à beaucoup plaira à l'utilisateur"**

-  Avantage : Fonctionne sans historique utilisateur
-  Limite : Pas personnalisé, favorise le mainstream
-  Usage : Fallback automatique pour nouveaux utilisateurs

Similarité de contenu (Embeddings)

```
Profil_user = mean(embeddings_articles_lus)  
Similarité = cosine_similarity(profil_user, all_articles)
```

Philosophie : **"Recommander ce qui ressemble à ce que l'utilisateur aime"**

-  Avantage : Très personnalisé, cohérence thématique
-  Limite : Cold start users, risque de bulle de filtrage
-  Usage : Utilisateurs avec historique riche (≥5 articles)

Clustering utilisateurs (K-means)

```
Clusters = KMeans(n=5).fit(user_features)  
Reco = popular_in_cluster(user.cluster_id)
```

Philosophie : **"Les utilisateurs similaires apprécient les mêmes articles"**

-  Avantage : Découverte, diversité, filtrage collaboratif
-  Limite : Nécessite historique, recalcul quotidien
-  Usage : Exploration au-delà des préférences habituelles

Approche hybride (Weighted mix)

```
Score = 0.4×clustering + 0.3×contenu + 0.2×popularité + 0.1×diversité  
if consensus: score += 0.1 # Bonus articles recommandés par plusieurs méthodes
```

Philosophie : **"Combiner les forces, compenser les faiblesses"**

-  Avantage : Robuste, équilibré, maximise la satisfaction
-  Limite : Plus complexe, paramètres à ajuster
-  Usage : Production, maximise les performances globales

🔥 Méthode 1 : Popularité normalisée par âge

🎯 Principe

"Recommander les articles avec le meilleur engagement relatif à leur âge, tout en donnant un coup de pouce aux nouveautés"

Philosophie : Un article récent avec 10 clics peut être plus "tendance" qu'un article ancien avec 100 clics.

📌 Formule :



```
# Score de base
score = (0.7 × unique_users + 0.3 × total_clicks) / article_age_months

# Boost nouveauté (cold start articles)
if age < 24h: score *= 1.5 # +50%
if age < 72h: score *= 1.2 # +20%
```



Message clé

Méthode simple et robuste, fondation du système de fallback pour 65% des utilisateurs.

✅ Avantages

-  Cold start résolu : Fonctionne sans historique utilisateur
-  Détection de tendances : Capture l'actualité brûlante
- ⚡ Performance optimale : <50ms (calcul simple)
- 🎯 Fallback universel : Base du système pour nouveaux users

⚠️ Limites

-  Peu personnalisé : Mêmes recommandations pour tous
-  Favorise le mainstream : Articles de niche moins visibles
- → **Solution :** Utilisé uniquement comme fallback automatique

🎯 Cas d'usage

1. Fallback automatique
2. Endpoint "Tendances" : GET /popular?limit=10
3. Boost nouveaux articles :
 - a. Article <24h : visibilité ×1.5 automatique
 - b. Garantit découvrabilité immédiate

Méthode 2 : Similarité de contenu (Embeddings)

Principe

"Recommander des articles similaires à ceux déjà lus par l'utilisateur"

Philosophie : "Si vous avez aimé X, vous aimerez Y" (cohérence thématique)

Architecture



```
# 1. Construire le profil utilisateur
user_profile = mean(embeddings_articles_lus) # Vecteur 250D

# 2. Calculer similarité avec tous les articles
similarities = cosine_similarity(user_profile,
all_articles_embeddings)

# 3. Recommander les Top 5
recommendations = top_N(similarities, n=5)
```



Fallback métadonnées (nouveaux articles)

- Si embeddings manquants : Utiliser catégories + longueur d'articles

Avantages

-  Fortement personnalisé : Basé sur les préférences réelles
-  Cohérence thématique : Articles similaires garantis
-  Explicable : "Similaire à l'article X que vous avez lu"
-  Précision élevée : Embeddings 250D (Sentence-BERT)

Message clé

Personnalisation maximale pour utilisateurs actifs, avec fallback robuste pour nouveaux articles.

Limites & Solutions

Limite	Impact	Solution implémentée
 Cold start utilisateurs	Besoin ≥5 articles lus	Fallback automatique → Popularité
 Embeddings volumineux	364MB à charger	Lazy loading (1ère requête)
 Bulle de filtrage	Manque de diversité	Compensé dans hybride (10% diversité)

Méthode 3 : Clustering utilisateurs (K-means)

Principe

"Regrouper les utilisateurs similaires et recommander ce qui plaît à leur segment"

Philosophie : "Les utilisateurs comme vous apprécient ces articles" (filtrage collaboratif)

Architecture

```
# 1. Extraire features comportementales (10+ variables)
user_features = [
    total_clicks, unique_articles, category_diversity,
    avg_words_preference, clicks_per_hour,
    top_10_categories_preferences # One-hot encoding
]

# 2. Clustering K-means (5 segments)
clusters = KMeans(n_clusters=5).fit(scaled_features)

# 3. Recommendations = articles populaires dans le cluster
cluster_id = get_user_cluster(user_id)
recommendations = popular_articles_in_cluster(cluster_id)
```

Avantages

-  Découverte (serendipity) : Articles hors du profil habituel
-  Filtrage collaboratif : Exploite la "sagesse de la foule"
-  Segmentation marketing : Insights business réutilisables
-  Scalable : Calcul en batch, cache persistant

Limites

-  Cold start users : Besoin ≥5 articles pour clustering
-  Mise à jour quotidienne : Recalculés tous les jours
-  Moins personnalisé : Recommandations au niveau segment
- Fallback automatique → Popularité si historique insuffisant

5 segments utilisateurs identifiés

Cluster	Profil type	Caractéristiques	% Users
0	 Lecteurs occasionnels	<10 clics, diversité faible	45%
1	 Lecteurs réguliers	10-50 clics, diversité moyenne	30%
2	 Power users	>50 clics, diversité élevée	15%
3	 Lecteurs rapides	Articles courts (<150 mots)	7%
4	 Lecteurs approfondis	Articles longs (>250 mots)	3%



Méthode 4 : Approche hybride

Principe

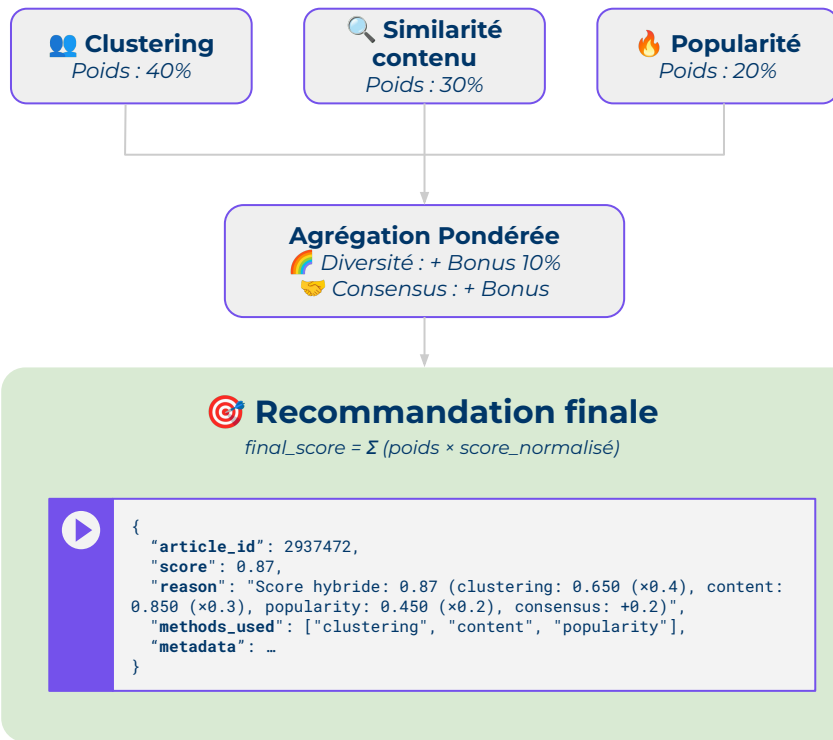
"Combiner les forces des 3 méthodes pour maximiser pertinence ET diversité"

Endpoint

- POST /recommend/{user_id}?method=hybrid

Gestion Intelligente

- Reason détaillée** : Indique le score de chaque méthode
- Méthodes utilisées** : Liste des méthodes réellement utilisées
- Diversité (10%)** : Bonus de 10% pour les articles de nouvelles catégories
- Consensus Bonus** : Plus un article est recommandé par plusieurs méthodes, plus il est favorisé
- Détecte le cold start** :
 - Vérifie si l'utilisateur a un historique suffisant
 - Fallback : Bascule automatique vers "popularité"





Résultats comparés

Critère	Popularité	Contenu	Clustering	Hybride
Use case optimal	Nouveaux users	Users actifs	Découverte	Production
Personnalisation	★ Faible	★★★★ Élevée	★★ Moyenne	★★★★ Très élevée
Diversité résultats	★ Faible	★★ Moyenne	★★★★ Élevée	★★★★ Très élevée
Cold start utilisateurs	✓✓✓ Parfait	✗ Impossible	⚠ Limité	✓✓ Excellent
Cold start articles	✓✓ Bon (boost)	⚠ Fallback requis	⚠ Indirect	✓ Bon (multi-source)
Données requises	Clics seulement	Embeddings 250D	Features users	Toutes
Complexité technique	● Simple	● Moyenne	● Élevée	● Très élevée
Temps de réponse	<50ms	~100ms	~80ms	~150ms
Mise à jour	Temps réel	Temps réel	Quotidienne	Temps réel



Gestion du Cold Start

🎯 Double défi

- Cold start utilisateurs : **Comment recommander sans historique ?**
- Cold start articles : **Comment rendre visibles les nouveaux contenus ?**

👤 Cold start utilisateurs



```
# Critère double : quantité ET qualité
has_sufficient_history = (
    unique_articles_read(user_id) >= 5 # Quantité
    AND all_articles_exist_in_metadata # Qualité
)
```

Métadonnées API



```
{
  "fallback_applied": true,
  "actual_method": "popularity",
  "fallback_reason": "Historique insuffisant (2 articles < 5 requis)"
}
```

Scénarii

Phase utilisateur	Articles lus	Méthode appliquée	Indicateur UI
Nouveau	0-4	🔥 Popularité (fallback auto)	🟡 Badge jaune
Établi	≥5	🧩 Hybride personnalisé	🟢 Badge vert

Comportement par méthode

Méthode demandée	User nouveau (<5 articles)	User établi (≥5 articles)
🔥 Popularité	✅ Exécution directe	✅ Exécution directe
📖 Contenu	⚠️ Fallback → Popularité	✅ Similarité embeddings
👥 Clustering	⚠️ Fallback → Popularité	✅ Articles du segment
🧩 Hybride	⚠️ Fallback → Popularité	✅ Combinaison complète





Cold start articles

3 stratégies complémentaires :

① Boost temporaire de nouveauté

```
if age < 24h: score *= 1.5 # +50%  
if age < 72h: score *= 1.2 # +20%
```

Exemple :

- Article récent (1 jour, 10 clics) : score = 450
- Article ancien (8 mois, 800 clics) : score = 92.5
- → Le nouvel article surpasse l'ancien de 5x ! ✓

② Endpoint dédié "Nouveautés"

```
GET /articles/recent?hours=48&category_id=281
```

Utilisation :

- Section "📰 Nouveautés" dans l'interface
- Indépendant des clics (visibilité garantie)
- Filtrage par catégorie optionnel

③ Fallback métadonnées Si embeddings manquants :

Utiliser catégories + longueur préférées

```
score_metadata = similarity(user_top_categories, article_category)  
+ similarity(user_avg_words, article_words)
```



Impact sur les méthodes

Méthode	Article avec clics	Article sans clics (nouveau)
🔥 Popularité	Score normal	Boost x1.5 compense le 0 clic
📖 Contenu	Similarité embeddings	Fallback métadonnées
👥 Clustering	Popularité segment	Popularité segment
🧩 Hybride	Combinaison complète	Bénéficie du boost via popularité



Cold Start : Tableau récapitulatif

Message clé

Double stratégie robuste garantissant recommandations pertinentes dès le premier jour (users ET articles).

Aspect	Nouvel utilisateur	Nouvel article
 Détection	<5 articles uniques lus	Âge <72h ou embeddings manquants
 Stratégie	Fallback → Popularité	Boost ×1.5 + Endpoint + Fallback métadonnées
 Visibilité	Immédiate (0ms)	Immédiate (0ms)
 Transition	Progressive (après 5 articles)	Progressive (après 3 jours)

Exemple utilisateur réel

User ID 1 (2 clics seulement) demande méthode Hybride :

1. Détection : unique_articles = 2 < 5 ❌
- ↓
2. Fallback automatique → Popularité
- ↓
3. Réponse API :
 - recommendations: [Top 5 articles populaires]
 - fallback_applied: true
 - actual_method: "popularity"
 - fallback_reason: "Historique insuffisant (2 < 5)"
- ↓
4. UI affiche badge  "Fallback appliqué"

Expérience utilisateur préservée :

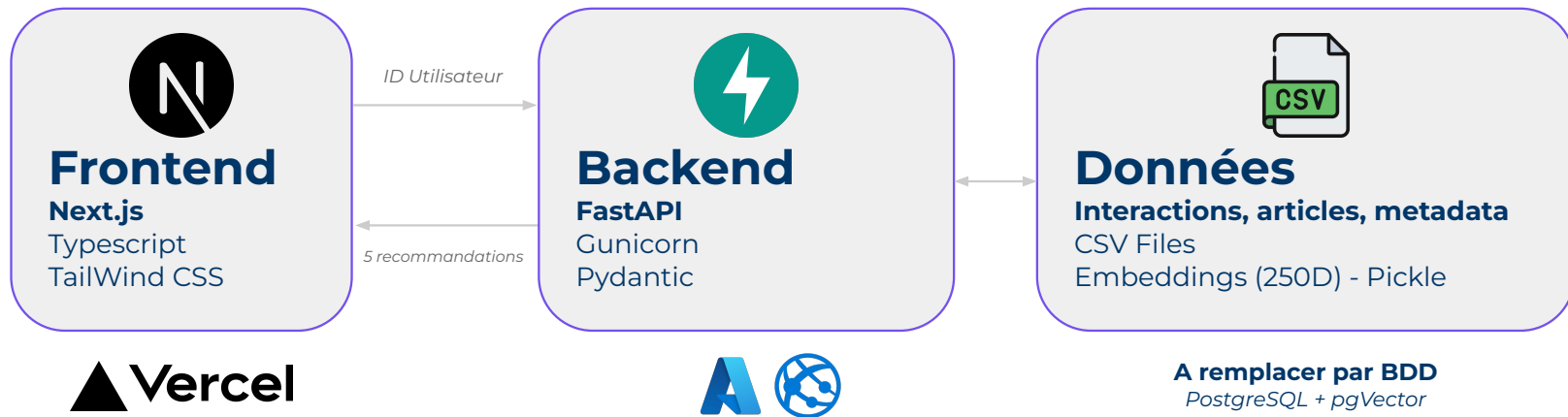
Toujours 5 recommandations pertinentes ✓

Message clé

Double stratégie robuste garantissant recommandations pertinentes dès le premier jour (users ET articles).



Architecture technique



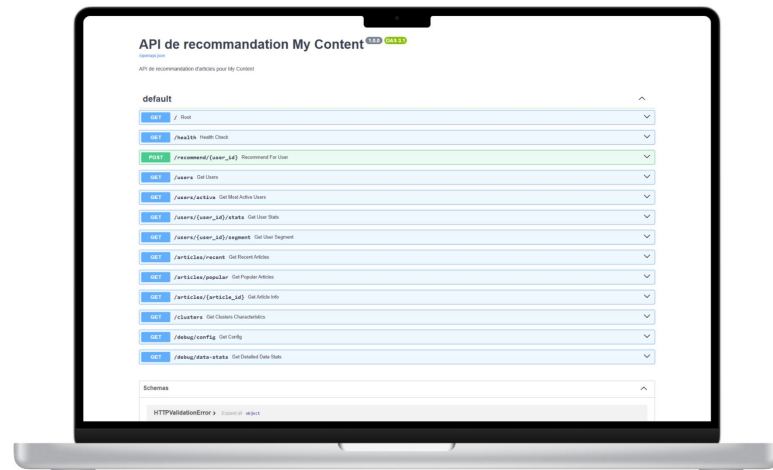
Backend - FastAPI

API REST - Endpoints principaux

Endpoint	Méthode	Description
/recommend/{user_id}	POST	5 recommandations (4 méthodes)
/users/active	GET	Top 20 utilisateurs actifs
/articles/popular	GET	Top 10 articles populaires
/articles/recent	GET	Nouveaux articles (<48h)
/health	GET	Santé de l'API + stats données
/debug/data-stats	GET	Statistiques détaillées

Réponse

```
{
  "user_id": 5890,
  "method": "hybrid",
  "fallback_applied": false,
  "recommendations": [
    {
      "article_id": 156789,
      "score": 0.847,
      "reason": "Score hybride: clustering(0.89) + content(0.76)",
      "metadata": { "category_id": 281, "words_count": 195 }
    }
  ]
}
```



Documentation interactive

Swagger UI : [/docs](#)

Déploiement Azure Functions (architecture serverless)

URL de production : <https://my-content-api-oc.azurewebsites.net/>



Frontend - Next.js

Technologies :

- **Next.js 15** : App Router, SSR/CSR hybride
- **TypeScript** : Typage statique complet
- **Tailwind CSS** : Styling utility-first
- **shadcn/ui** : Composants accessibles (Card, Tabs, Select, Badge)

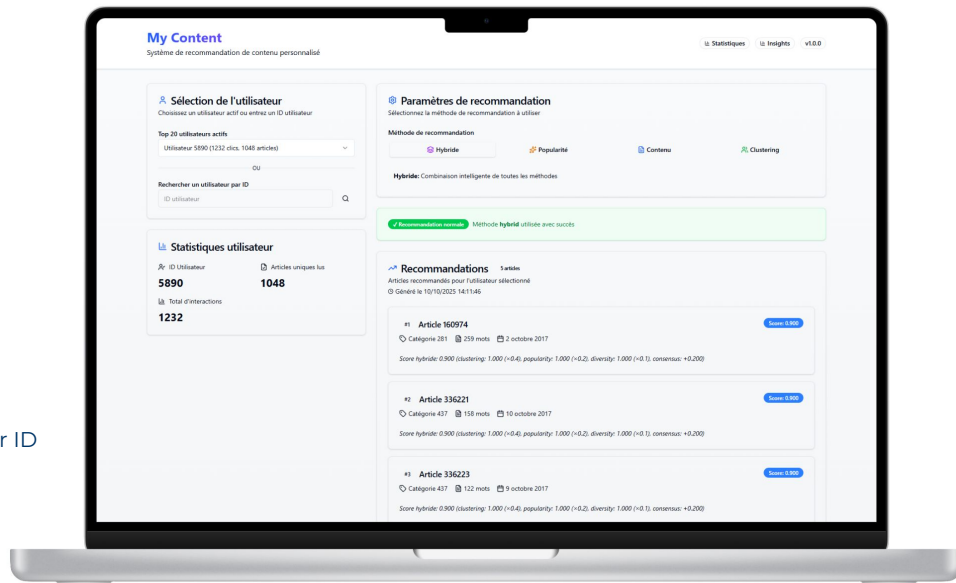
3 pages principales :

- [/](#) - Recommandations personnalisées
- [/insights](#) - Articles populaires & récents
- [/statistics](#) - Vue d'ensemble dataset + top users

Fonctionnalités clés :

L'interface utilisateur permet de :

- **Sélectionner un utilisateur** parmi les plus actifs ou rechercher par ID
- **Choisir la méthode de recommandation**
 - Hybride, Popularité, Contenu, Clustering
- **Visualiser les 5 articles recommandés** avec leurs métadonnées
- **Indicateur fallback temps réel** (badge vert/jaune)
- **Consulter les statistiques utilisateur** en temps réel
- **Accéder aux insights** (articles populaires et récents)





Déploiement de l'API sur Azure



L'API backend est déployée en production et pleinement fonctionnelle sur Azure !*

URL de production

<https://my-content-api-oc.azurewebsites.net/>

Infrastructure

- ✓ Plan : **PIV2 Premium** (1 core, 3.5GB RAM, ~150€/mois)
- ✓ Région : France Central
- ✓ Runtime : Python 3.11
- ✓ Workers Unicorn : 2
- ✓ Timeout : 120s
- ✓ CORS configuré

Évolution des plans testés

- B1 Basic** (13€/mois) : ✗ Insuffisant - Workers killed (OOM)
- S1 Standard** (60€/mois) : ⚠ Limite - Timeouts sur certains endpoints
- PIV2 Premium** (150€/mois) : ✓ Optimal - Stable et performant

my-content-api-oc Application web

Parcourir Arrêter Échanger Redémarrer Supprimer Actualiser Télécharger le profil de publication Réinitialiser le profil de publication

Consider using API Management Service since your app has an OpenAPI configuration or a swagger page.

Bases

Groupe de ressources (...)	my-content-rg	Domaine par défaut	my-content-api-oc.azurewebsites.net
État	Exécution	Plan App Service	davidscanu14_asp_0798 (P1v2: 1)
Emplacement (déplacer)	France Central	Système d'exploitation	Linux
Abonnement (déplacer)	Abonnement Azure 1	Vérification de l'intégrité	Non configuré
ID d'abonnement	26e96eaa-900d-4a6b-a71f-5893ee95e876		

Étiquettes (modifier) Ajouter des étiquettes

Propriétés Supervision Journaux Fonctionnalités Notifications (1) Recommandations

Application web

Nom	my-content-api-oc
Modèle de publication	Code
Pile d'exécution	Python - 3.11

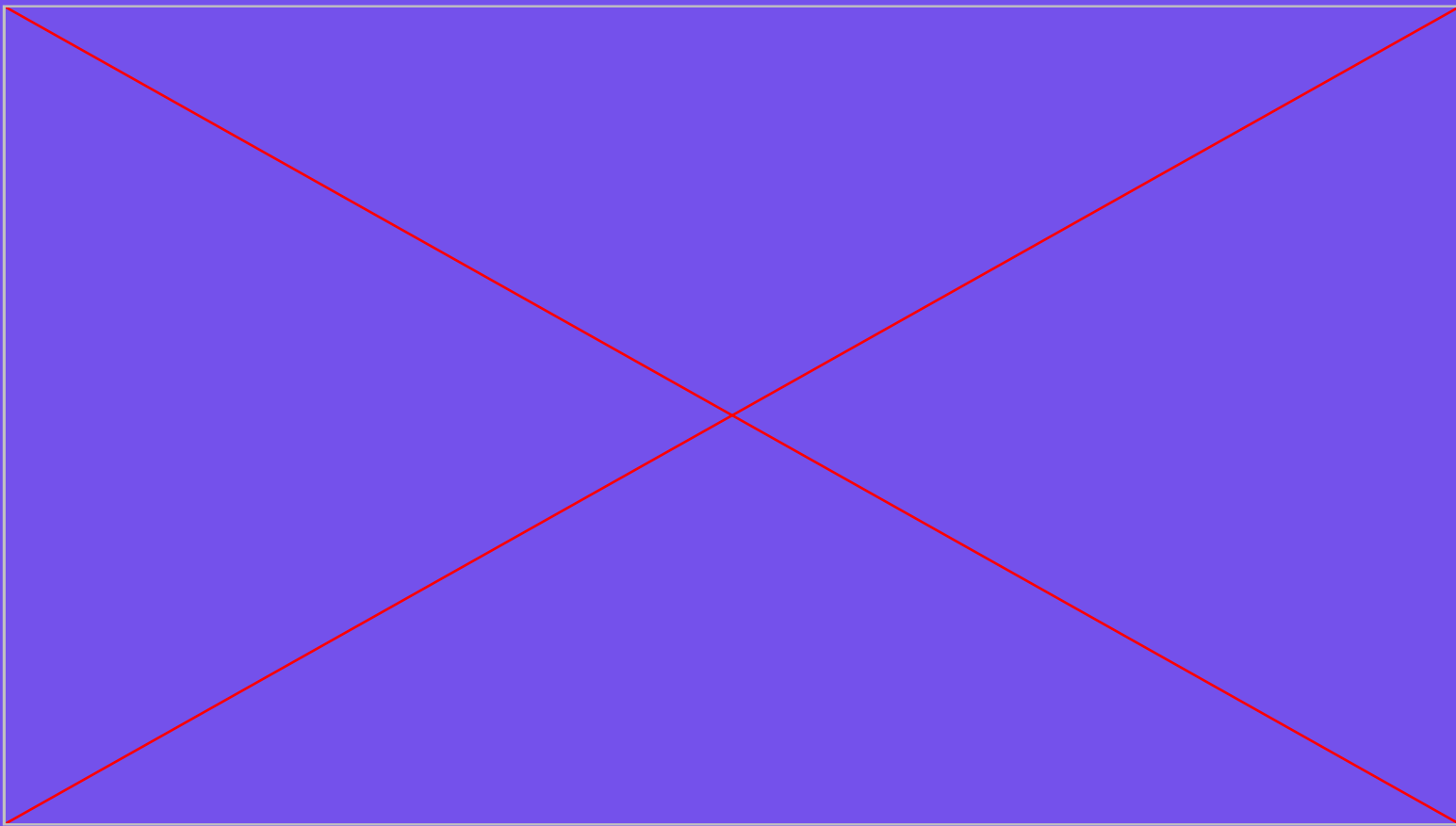
Domaines

Domaine par défaut	my-content-api-oc.azurewebsites.net
Domaine personnalisé	Ajouter un domaine personnalisé

Hébergement

Type d'offre	Plan App Service
Nom	davidscanu14_asp_0798
Système d'exploitation	Linux
Nombre d'instances	1
Référence SKU et taille	PremiumV2 (P1v2) Augmentez la mise à l'échelle





p10-video-demo-my-content-app.mp4



Architecture cible

Pipeline pour embeddings

- Encodage des embeddings d'articles avec **Transformers**

Base de données

- Remplacer CSV par base de données **PostgreSQL**
- Stockage des embeddings avec **pgVector**

Pipeline de d'intégration et déploiement continu

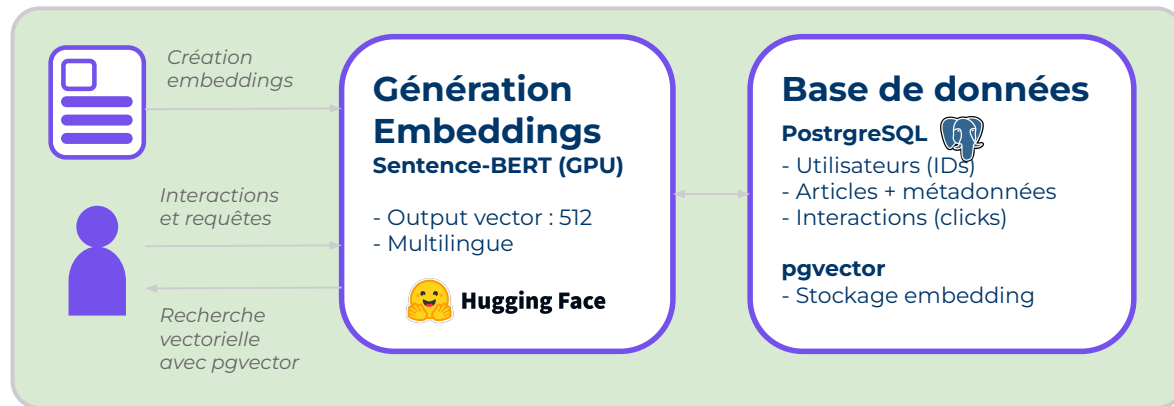
- CI/CD : GitHub Actions (tests auto, déploiement continu)

Monitoring avancé

- Prometheus + Grafana

Nouveaux Articles

Nouveaux Users





Bilan & Livrables



Réalisations MVP

MVP complet et fonctionnel, code professionnel versionné, architecture scalable définie. Prêt pour beta-test et déploiement production.

Système de recommandation complet :

-  **4 méthodes implémentées :**
 - Popularité
 - Contenu
 - Clustering
 - Hybride
- **Cold start résolu :**
 - Utilisateurs (fallback auto)
 - Articles (boost $\times 1.5$)

Application fonctionnelle :

- **Frontend Next.js** : 3 pages interactives (Reco, Insights, Stats)
- **Backend FastAPI** : API REST + Swagger docs
- **Déploiement en production** : Serverless Azure Functions



Points forts du système

Innovation technique :

-  **Approche hybride** : Pondération intelligente (40/30/20/10)
-  **Fallback automatique** : Transparent et expliqué
-  **Embeddings 250D**
-  **Clustering K-means** : 5 segments comportementaux

Architecture évolutive :

- **Architecture cible définie** : PostgreSQL + pgvector
- **Pipeline embeddings** : encodage des nouveaux articles
- **Pipeline de d'intégration et déploiement continu** (CI/CD)
- **Monitoring avancé** (Prometheus + Grafana)



Merci 🙏

Avez-vous des questions ?



Livrables

- [Dépôt GitHub : oc-ai-engineer-p10-realisez-application-recommandation-de-contenu](#)



Vidéo de démonstration de l'application

- [p10-video-demo-my-content-app.mp4](#)



A propos

- [David Scanu - Développeur en intelligence artificielle - Carrefour | LinkedIn](#)
- [Formation AI Engineer - OpenClassrooms](#)